



Exercise 06: Saturation & IR Sensor Math

Part 1: Saturation

- With digital control schemes, control input decisions are made based on sensor inputs. For our MazeRunner robot following heading, a MEMs gyro provides the current heading to **12-bit precision** and covers a **360°** range.
- However, if it differs from our desired heading by **90° or more** we are not going to do anything different than drive one wheel hard forward and the other wheel hard back to correct course.
- We can reduce the width of the data coming from the gyro and not lose any “needed” information.
 - By reducing to a smaller number of bits, we can make down stream calculations narrower (fewer bits wide) and thus use fewer gates, and less power.
- You are going to write **dataflow Verilog** to saturate the **error** term, and implement the **P_term** of the PID controller.





Exercise 06: Saturation & IR Sensor Math

Saturation examples: 8-bit signed number → 5-bit signed number

Ex 1: **01010110**

MSB=0 → +ve

greatest +ve number in 5-bits? **01111**

01010110 > **01111** since bits [6:4] has 1s

saturate to **01111**

Ex 2: **11010110**

MSB=1 → -ve

greatest -ve number in 5-bits? **10000**

11010110 < **10000** since bits [6:4] has 0s

saturate to **10000**

Ex 3: **11110110**

MSB=1 → -ve

greatest -ve number in 5-bits? **10000**

11110110 > **10000** since bits [6:4] has no 0s

keep lower bits **10110**

Pseudo code:

```
if ((number is negative) &&
    (any upper bits (in certain range) are zero)) {
    saturate to most negative number
}
else if ((number is positive) &&
    (any upper bits (in certain range) are one)) {
    saturate to most positive number
}
else {
    number not out of range so just copy over lower bits
}
```



Exercise 06: Saturation & IR Sensor Math

- Implement saturation using **dataflow Verilog** with the following interface

Signal	Dir	Description
error[11:0]	in	12-bit signed error term (heading – desired_heading)
P_term[13:0]	out	14-bit signed P component of PID controller

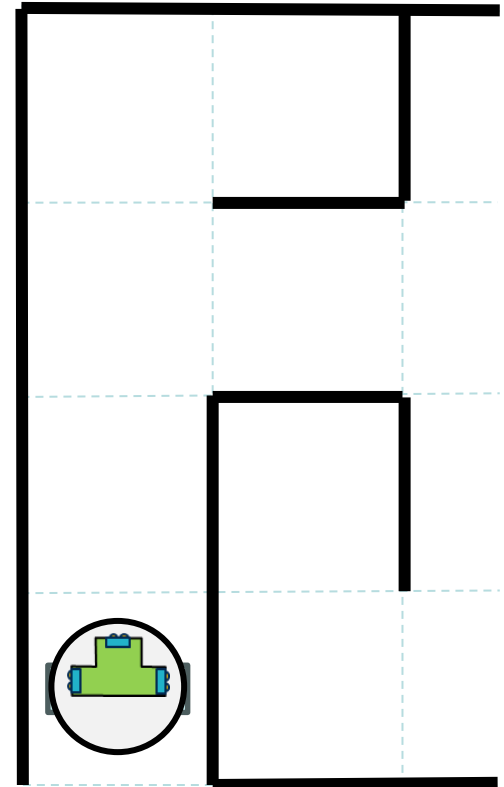
- You should use **assign** statements with **ternary** operator to implement conditionals
- Submit the design as **P_term.sv**
- Create a self-checking testbench → **P_term_tb.sv**
 - You should test several values (saturating both directions and not saturating)
- Part 1 is a 20pt question in HW2



Exercise 06: Saturation & IR Sensor Math

Part 2: IR Sensor Math

- The MazeRunner has 3 IR sensors
 - One pointing forward to detect a wall in front
 - Left and right sensors will 1) seek openings and 2) correct for gyro drift
- Gyro reading will drift over time
 - MEMs gyro provides angular rate (degrees/sec)
→ integrate to get angular position
 - Small error will add up over time & heading becomes inaccurate
- We need to use the IR sensor reading to correct gyro drift
- In this part, you will use **dataflow** Verilog to describe a provided circuit that handles gyro drift correction

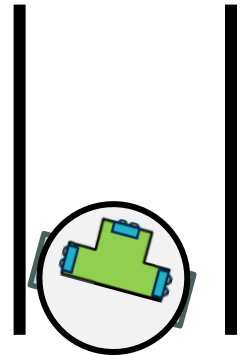
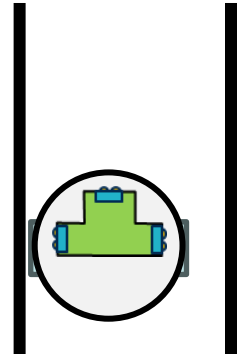




Exercise 06: Saturation & IR Sensor Math

There are 2 scenarios where left/right IR sensors help correct

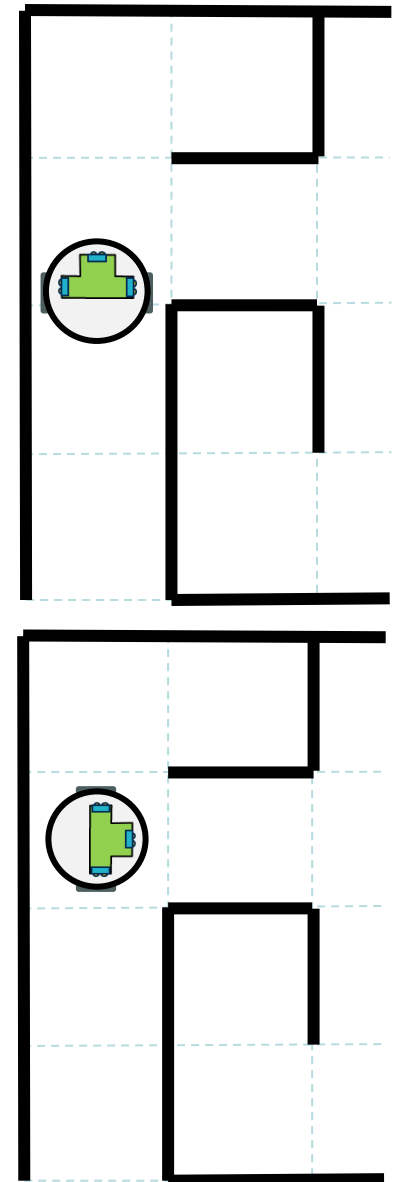
- Too far to one side
 - If the MazeRunner is too close to a wall as shown, the left IR sensor will read significantly lower than the right
 - A **P**-term (proportional) of IR sensors will help us correct this
 - Want to adjust towards right
- Direction of the robot is off and it's veering towards a wall
 - Left sensor will read lower than right, but we don't want to adjust more right
 - The **D**-term (time rate of change of the IR sensors) can help us correct this.





Exercise 06: Saturation & IR Sensor Math

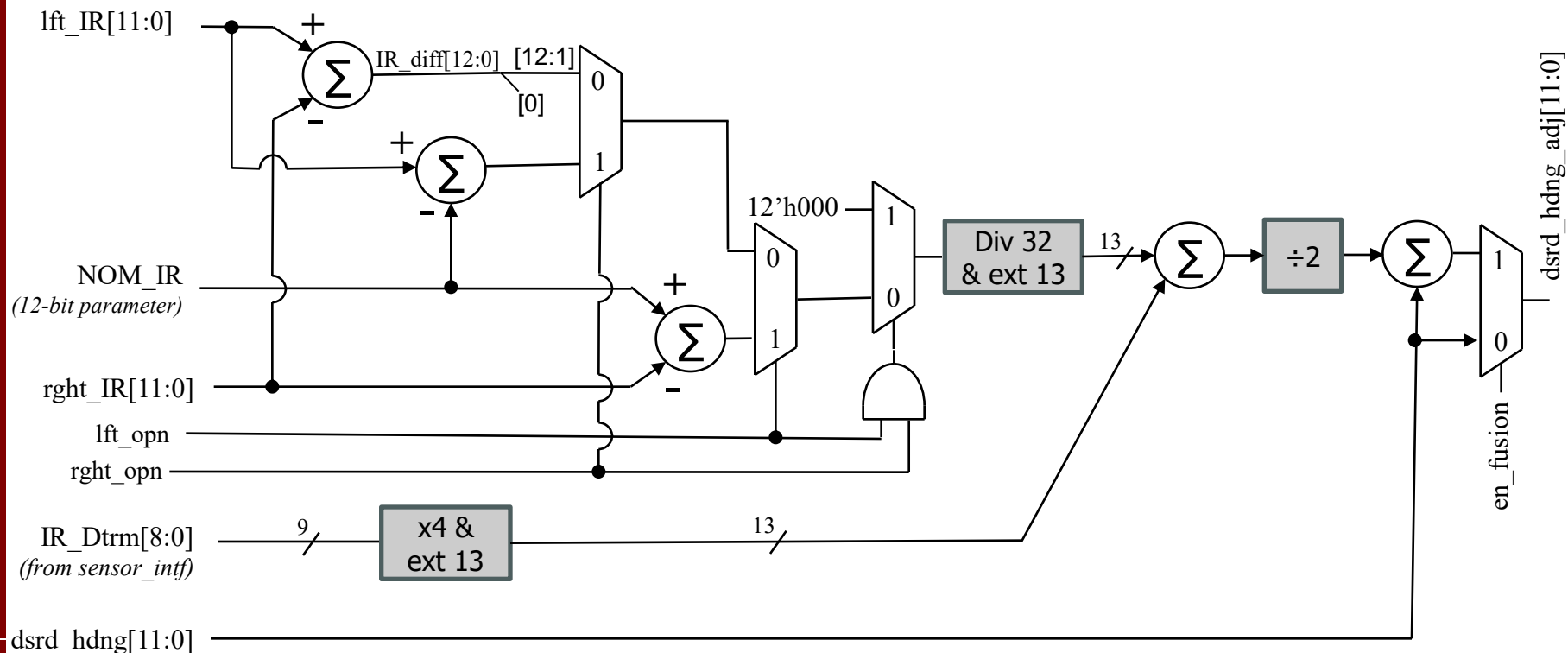
- Navigation is best done using the difference of the left vs right IR sensors
- However, depending on the MazeRunner's position in the maze both sensors may not be available ...
- If right sensor is not available, run with just info from the left IR sensor
 - Take difference between left & nominal IR reading (parameter)
- If neither are available, use only gyro-based heading





IR Math design

Signal	Dir	Description
lft_opn, right_opn	in	Indicate left/right IR sensors have no reading (the path is open)
lft_IR[11:0], right_IR[11:0]	in	IR readings (decrease closer object (maze wall) is)
NOM_IR	param	12-bit parameter defaulted to 12'h900
IR_Dtrm[8:0]	in	Derivative of IR readings, comes from <i>sensor_intf</i> block
en_fusion	in	IR fused with gyro only when moving at decent speed
dsrd_hdng[11:0]	in	Desired heading of MazeRunner (comes from <i>cmd_proc</i>)
dsrd_hdng_adj[11:0]	out	What we are making – the adjusted desired heading





Exercise 06: Saturation & IR Sensor Math

- Use **dataflow** Verilog to create the specified design → **IR_Math.sv**
- A self-checking testbench (***IR_Math_tb.sv***) is provided. Debug and submit as much as you have completed by end of class. This is problem on HW2.
- Part 2 is a 20pt question in HW2